

Know VAT & ZATCA — Online Inference Testing

Issues found, fixes applied, and working commands — 2026-07-23 (follow-up to the offline ingestion pipeline session)

Contents

- 1. Context
- 2. What already existed (verified before testing)
- 3. Problems found and fixed (5)
- 4. Known open issue — Modal generation reliability
- 5. Working commands reference
- 6. Final verified result

1. Context

With the offline ingestion pipeline fixed and verified (see [docs/know_vat_zatca_ingestion_pipeline_fixes.pdf](#)), this session moved to testing the **online** half: the FastAPI endpoint, the LangGraph retrieval+generation pipeline, and the Modal-hosted embedding/generation models it calls. Tested through the real stack — `docker compose` + Swagger UI + direct `curl` — not just Python-level unit checks.

2. What already existed (verified before testing)

Before changing anything, confirmed the online code was already fully built:

- **API endpoint:** `POST /api/know-vat-zatca/chat` in `app/main.py`, JWT-protected.
- **LangGraph pipeline:** `agents/know_vat_n_zatca/online/graph.py` — detect language → jurisdiction check → embed query → search Qdrant (same-language-first, cross-language fallback) → resolve table pointers + cross-references → generate via Modal Qwen-lite → trim memory. Postgres-backed session checkpointing.
- **Modal embedder:** a real, already-deployed BGE-M3 `/embed` endpoint in `modal-serving/modal_app.py` (the same one the offline pipeline uses), request/response shape matching exactly what `retrieval.py` expects.

Nothing needed to be built from scratch — this session was entirely about wiring gaps and retrieval-quality bugs, not missing functionality.

3. Problems found and fixed

1 MIZAN_EMBEDDER_URL missing from deployment config

Read correctly in `agents/know_vat_n_zatca/config.py` and already set in the local `.env`, but **not documented in** `.env.example` and **not passed through in** `docker-compose.yml`'s `chatbot` service — only `MIZAN_QWEN_LITE_URL` was listed there. Compose only forwards env vars it's explicitly told about, so

running via `docker compose up` would leave `MODAL_EMBEDDER_URL` empty inside the container and every request would fail immediately with `ModalEndpointError("Modal endpoint URL is not configured")`.

FIX

Added `MIZAN_EMBEDDER_URL: ${MIZAN_EMBEDDER_URL:-}` to `docker-compose.yml`'s `chatbot` environment block, and documented it in `.env.example` alongside `MIZAN_QWEN_LITE_URL`.

2 Docker container-name conflict + network gap

`docker compose up` failed with *"Conflict. The container name /mizan-qdrant is already in use"* — a `qdrant` container was already running (holding all 1,243 ingested points from the offline session) but was never started by this compose project, so compose didn't recognize it as "its" `qdrant` and tried to create a new one under the same name.

WHY NOT JUST REMOVE THE OLD CONTAINER

Doing so and letting compose create a fresh one would have attached a **new, empty, project-prefixed volume** (e.g. `mizanai_qdrant_data`) instead of the real `qdrant_data` volume holding the actual knowledge base — it would have come up looking fine but silently empty.

Separately: even once running, the existing container wasn't on the `mizanai_default` network the `chatbot` container uses, so `http://mizan-qdrant:6333` wouldn't have resolved between them anyway (same network gap already documented in the offline pipeline PDF).

FIX — KEEP THE EXISTING CONTAINER, CONNECT IT TO THE RIGHT NETWORK, TELL COMPOSE TO LEAVE IT ALONE

```
docker network connect mizanai_default mizan-qdrant
docker compose up -d chatbot --no-deps
```

Confirmed via logs: `GET http://mizan-qdrant:6333/collections "HTTP/1.1 200 OK"` — reaching the real data, not an empty fresh instance.

3 Table content wasn't being retrieved at all

Real test query: *"What are the penalties for late VAT registration in Saudi Arabia?"* — the actual answer ("Failure to apply for VAT registration" → "SAR 10,000") sat in the `VAT Penalties (Taxation Violation Fines)` table the whole time, but never appeared in retrieval results.

ROOT CAUSE, PROVEN WITH REAL NUMBERS

Table-pointer chunks only embed a placeholder description — column names and row count, not the actual content:

```
"A table from VAT Penalties (Taxation Violation Fines) with 10 rows,
containing the following columns: violation_description, penalty."
```

Measured its cosine similarity against the real query using the live Modal embedder:

Description version	Similarity to query	Clears 0.5 relevance threshold?
Old (columns only)	0.452	No — filtered out before ranking even mattered
New (see fix below)	0.552	Yes — competitive with real prose results (0.55–0.66)

FIX — AGENTS/KNOW_VAT_N_ZATCA/OFFLINE/TABLE_DESCRIPTIONS.PY, DETERMINISTIC (NO ANTHROPIC API CALL, PER STANDING INSTRUCTION)

Rewrote `generate_placeholder_description()` to fold in actual row content — up to `max_chars` (default 1500) of real cell values, not just column names:

```
"A table from VAT Penalties (Taxation Violation Fines) with 10 rows,
containing the following columns: violation_description, penalty.
Table content: violation_description: Submit false documents to evade
the payment or reduce VAT due or unlawfully obtain refunds | penalty:
At least VAT due, up to 3x the goods/service value; violation_description:
Failure to apply for VAT registration | penalty: SAR 10,000; ..."
```

Small tables (the common case) get their entire content embedded; large tables get a bounded leading sample. `ingest.py` updated to pass `rows` instead of just `row_count`. Table-pointer point IDs are keyed by `table_ref`, not content, so re-running ingestion safely **updates** the description/embedding in place — no wipe needed, unlike the offline session's chunking-algorithm fixes.

4 Even when retrieved, a table match could still be dropped from context

`resolve_and_cap()` capped the final context at `CONTEXT_CHUNK_CAP` (6) by simple score order across all retrieved chunks. A table's placeholder description tends to score lower than real prose passages even when it's the more directly useful source — so a table that legitimately passed the relevance threshold could still rank 7th–10th among 10 candidates and get silently truncated away.

FIX — AGENTS/KNOW_VAT_N_ZATCA/ONLINE/RETRIEVAL.PY

`resolve_and_cap()` now puts resolved table matches **first**, ahead of prose, before applying the cap — guaranteeing any table that cleared the relevance threshold survives into the final context, with cross-reference resolutions still filling in last as before.

Verified with a synthetic test: a table match placed last (position 8 of 8) still made it into the final 6, ahead of 5 prose chunks:

```
content_kinds in final context: ['table_rows', 'prose', 'prose', 'prose', 'prose', 'prose']
```

5 Testing gotcha: Arabic text mangled by inline shell arguments

Not a pipeline bug — a test-tooling trap worth documenting so it doesn't get mistaken for one again. Passing Arabic query text inline via `curl -d '{"message": "..."}'` in Git Bash on Windows silently corrupted the UTF-8 bytes before they reached the server, causing a real Arabic query to come back as a false `fallback_kind: "nothing_found"` with zero citations — looking exactly like a retrieval failure.

FIX — ALWAYS SEND NON-ASCII PAYLOADS FROM A FILE

```
# WRONG on Windows/Git Bash -- mangles non-ASCII text:
curl -d '{"message": "ما هي..."}' ...

# RIGHT -- write to a UTF-8 file first, then reference it:
curl --data-binary "@query.json" -H "Content-Type: application/json; charset=utf-8" ...
```

Re-running the identical query this way immediately surfaced the correct citations (XML Implementation Standard , Security Features Implementation Standard) — confirming the pipeline itself was fine all along.

4. Known open issue — Modal generation reliability

NOT FIXED THIS SESSION — INFRA-SIDE, OUTSIDE THIS REPO'S CODE

The Qwen-lite generation endpoint (`modal-serving/modal_app.py` 's generation class) failed on **4 of 9** real end-to-end test calls this session, always with the identical error:

```
Modal endpoint call failed: https://kalimi03--mizan-models-qwenlite-generate-
lite.modal.run:
('Connection aborted.', RemoteDisconnected('Remote end closed connection without
response'))
```

The client code (`agents/common/modal_client.py`) is doing the right thing — correct payload, generous 600s timeout, fails closed gracefully into a safe "couldn't find an answer" message rather than crashing or hallucinating. The connection is being dropped by Modal's side (or a gateway in front of it) before any response, consistently — this needs to be looked at directly in the Modal dashboard/logs for the `mizan-models` app, which this environment doesn't have CLI access to.

Separately, on the **2 of 9** calls that succeeded with Arabic output, one showed garbled leading tokens (stray Chinese characters, a nonsense fragment) before self-correcting into a coherent, correct answer — a distinct model-output-quality issue, not a connectivity one.

Also noted, not fixed: citations aren't deduplicated — a document cited by multiple retrieved chunks appears once per chunk in the response's `citations` list (e.g. the same document appeared 4 times in one response). Cosmetic, in `agents/know_vat_n_zatca/online/graph.py`'s `_generate_answer_node`.

5. Working commands reference

5.1 Bring the full stack up

One-time, only needed if the pre-existing `mizan-qdrant` container isn't already on the right network:

```
docker network connect mizanai_default mizan-qdrant
```

Then, every time you need to (re)build and start the API (add `--no-deps` whenever `qdrant/postgres` are already running outside this compose invocation, to avoid the naming conflict in §3.2):

```
docker compose up -d --build chatbot --no-deps
```

Confirm clean startup:

```
docker compose logs -f chatbot
# look for:
# INFO:agents.chatbot.langgraph_chatbot:Qwen Lite endpoint configured: True
# INFO:httpx:HTTP Request: GET http://mizan-qdrant:6333/collections "HTTP/1.1 200 OK"
# INFO:app.main:Service is up.
```

GOTCHA

`docker compose restart chatbot` reuses the existing image — it will **not** pick up code changes. Use `docker compose up -d --build chatbot --no-deps` instead whenever Python source under `agents/` or `app/` has changed.

5.2 Test via Swagger UI

1. Open `http://localhost:8000/docs`.
2. Expand **POST /api/auth/register** (or **/api/auth/login** if the account already exists) → Try it out → body:

```
{"email": "test@example.com", "password": "TestPass123!", "nickname": "Test"}
```

→ Execute. Copy the `jwt_token` from the response.

3. Click **Authorize** (padlock, top right) → paste the raw token (no `Bearer` prefix — Swagger adds it) → Authorize → Close.
4. Expand **POST /api/know-vat-zatca/chat** → Try it out → body:

```
{"message": "What is the mandatory VAT registration threshold in Saudi Arabia?"}
```

→ Execute. Reuse the response's `session_id` in a follow-up request to continue the same conversation thread.

5.3 Test via curl (scriptable, e.g. for regression checks)

```
# Register once, reuse the token for subsequent calls
TOKEN=$(curl -s -X POST http://localhost:8000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"claude-test@example.com","password":"TestPass123!","nickname":"Test"}' \
  | python -c "import json,sys; print(json.load(sys.stdin)['jwt_token'])")

# English / ASCII queries: inline -d is fine
curl -s -X POST http://localhost:8000/api/know-vat-zatca/chat \
  -H "Content-Type: application/json" -H "Authorization: Bearer $TOKEN" \
  -d '{"message":"What are the penalties for late VAT registration in Saudi Arabia?"}'

# Arabic / non-ASCII queries: MUST go through a file (see problem #5 above)
echo '{"message": "ما هي متطلبات إصدار الفاتورة الإلكترونية المبسطة؟"}' > query.json
curl -s -X POST http://localhost:8000/api/know-vat-zatca/chat \
  -H "Content-Type: application/json; charset=utf-8" -H "Authorization: Bearer $TOKEN" \
  --data-binary "@query.json"
```

5.4 Test retrieval in isolation (no LLM, no HTTP — fastest loop for retrieval-quality iteration)

```
python -c "
from dotenv import load_dotenv
load_dotenv()
from agents.know_vat_n_zatca.online.retrieval import embed_query, search_with_fallback

vector = embed_query('What are the penalties for late VAT registration?')
chunks, used_fallback = search_with_fallback(vector, 'en')
print(f'chunks found: {len(chunks)}, cross_language_fallback: {used_fallback}')
for c in chunks:
    p = c['payload']
    print(f"  score={c['score']:.3f} doc={p.get('document_type')} kind=
{p.get('content_kind')}\n")
"
```

This is how the table-description fix in §3.3 was validated before touching any live data — embed both the old and new description text directly and compare cosine similarity against a real query.

5.5 Check what a table's live embedded description actually says

```
curl -s -X POST http://localhost:6333/collections/know_vat_n_zatca/points/scroll \
-H "Content-Type: application/json" \
-d '{"filter": {"must": [{"key": "document_type", "match": {"value": "VAT Penalties (Taxation Violation Fines)"}]}}, "limit": 5, "with_payload": true}'
```

5.6 Diagnose a generation failure

```
docker compose logs chatbot --since=5m 2>&1 | grep -i "answer generation failed"
```

`fallback_kind: null` in a response combined with real (non-empty) `citations` means retrieval succeeded but the Modal generation call failed — check this log line for the exact error. `fallback_kind: "nothing_found"` with empty citations means retrieval itself found nothing above threshold — a different, retrieval-side problem.

5.7 Re-ingest after a `table_descriptions.py` or `chunker.py` change

Same command pattern as the offline pipeline session (full reference in [docs/know_vat_zatca_ingestion_pipeline_fixes.pdf §5.2](#)) — bind-mount `agents/` and the manifest directly so the container always runs current code:

```
MSYS_NO_PATHCONV=1 docker run --rm --gpus all \
  -e QDRANT_URL=http://host.docker.internal:6333 \
  -v mizanai_know_vat_zatca_sqlite:/data \
  -v "C:\path\to\models\bge-m3:/models/bge-m3" \
  -v "\agents:/app/agents" \
  -v "\docs\rag_ingestion_data\manifest.json:/app/docs/rag_ingestion_data/manifest.json" \
  know-vat-zatca-offline \
  python -m agents.know_vat_n_zatca.offline.ingest \
    --manifest docs/rag_ingestion_data/manifest.json \
    --embedding-model-path /models/bge-m3
```

Safe to re-run any time — table-pointer IDs are keyed by `table_ref` (upserts in place), and `insert_table_rows()` deletes-before-inserting, so this never duplicates data.

6. Final verified result



Auth (register/login/JWT)



Retrieval (EN + AR + cross-lang fallback)



Table-pointer resolution



Multi-turn session memory



Modal generation (flaky, infra-side)

Closing test — the exact query that started this investigation, run against the fully fixed pipeline through the real HTTP API:

Query: "What are the penalties for late VAT registration in Saudi Arabia?"

Reply: "The penalty for failure to apply for VAT registration is SAR 10,000.
This information is sourced from the VAT Penalties (Taxation Violation Fines)
document available on the ZATCA website, which is current as of the latest update."

Citations include: VAT Penalties (Taxation Violation Fines)

That figure — **SAR 10,000** — is the real value from the real table row, not a paraphrase or a guess. Confirms both fixes (§3.3 and §3.4) working together end-to-end, through the actual production code path (HTTP → auth → LangGraph → Qdrant → Modal → SQLite → Modal → response).

Generated 2026-07-23 — Mizan.ai, Know VAT & ZATCA (RAG Feature A) online inference pipeline. Companion to `know_vat_zatca_ingestion_pipeline_fixes.pdf`.